

International Symposium on Green Technologies and Applications (ISGTA'2023)

Machine Learning Algorithms for Intrusion Detection in IoT

Prediction and Performance Analysis

Ennaji Elmahfoud ^a, Salah Elhajla ^a, Yassine Maleh ^a, Soufyane Mounir ^a

^aLaSTI, Sultan Moulay Slimane University, Beni Mellal, 23000, Morocco

Abstract

The rapid growth of the Internet of Things (IoT) has created a large attack surface for adversaries to launch destructive cyber attacks. Intrusion Detection Systems (IDS) can be used to identify and monitor anomalies and threats in the IoT system. In this paper, we compare the performance of various machine learning algorithms for building IDS, including decision tree (DT), random forest (RF), k-Nearest Neighbor (k-NN), Ada Boost, and support vector machine (SVM), using the IoTID20 dataset. This dataset has three target classes, including a binary class for normal or abnormal behavior and classes for categories and sub-categories of the binary class. We select the most relevant features to minimize execution time and improve accuracy. Our results show that the decision tree algorithm generally achieved the highest accuracy at 99.80% with the lowest error rate compared to the other algorithms.

© 2024 The Authors. Published by ELSEVIER B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the International Symposium on Green Technologies and Applications

Keywords: IoT; Machine learning; Intrusion detection;

1. Introduction

The Internet of Things (IoT) refers to a network of interconnected devices that can communicate with each other without human intervention. These devices can be found in a wide range of industries such as transportation, farming, and healthcare, and offer numerous benefits including increased efficiency and resource conservation. The applications of IoT are changing the way we live and work by saving time and resources.

However, the IoT has become a top target for cybercriminals. There are many issues associated with IoTs, such as trust and privacy, data protection, network security and power management. To protect these devices from adversaries and unauthorized access or tampering, Intrusion Detection Systems (IDS) can be used to analyze the

activity of a computer or network system [1] and detect attempts at attack. An Intrusion Prevention System (IPS) [1] is a system that can take further action based on the detection of an attack. The challenge with IDS is to detect attacks in real-time within high-speed network traffic data.

An Intrusion Detection System can also employ the following detection techniques: Signature-based and Anomaly-based [2]. A Signature-based Detection technique works by searching network or host system traffic for specific byte patterns known to be malicious. The signatures or patterns are formed using rules based on traffic stream behavior. An Anomaly-based detection technique is used primarily to detect and classify new or novel attacks based on the traffic stream with the help of Machine Learning techniques [2]. With the increase in zero-day attacks and the complexity used by attackers, anomaly-based IDS are particularly well-suited to the current environment. Machine-learning play a key role in the development of anomaly-based intrusion detection systems. Classification is the frequently used technique in these systems to detect anomalies and make predictions on collected data.

In this paper, we conduct a performance comparison of five machine learning algorithms - Decision Tree (DT) [3], Support Vector Machine (SVM) [4], Random forest (RF) [5], Ada Boost [6], and k Nearest Neighbors (k-NN) [7] - on the IoT attack dataset. Our goal is to determine which algorithm is the most accurate in classifying the IoT 2020 network intrusion dataset. To achieve this, we establish a machine learning model to analyze and detect anomalies in the IoT dataset using selected features of high importance.

The rest of this paper is organized as follows. Section 2 is related work, followed by dataset description and pre-processing in Sect.3. In Sect.4 we present the methodology, algorithm implementation, and results in the evaluation. Finally, in Sect. 5 we conclude the paper.

2. Related work

Several studies have been conducted on IoT attack datasets to classify anomalies and threats. Many of them show good classification accuracy.

Hassan et al. [8] applied several machine learning approaches to predict attacks and anomalies on IoT devices. These approaches included logistic regression (LR), decision tree (DT), support vector machine (SVM), random forest (RF), and artificial neural network (ANN). The results indicated that RF, DT, and ANN performed better than the other methods, achieving an accuracy of 99.4%.

Jingyi Su et al. [9] applied multiple machine learning models to predict network attacks on IoT devices. The models tested included the random forest, decision tree, and gradient-boosting machine (GBM). The results showed that the decision tree algorithm had the highest accuracy, while the random forest had better AUC scores.

Reddy et al. [10] proposed a detection approach for protecting infrastructure on the fog node using the XGBoost ensemble method and Exact Greedy Boosting for fine-tuning hyperparameters. The proposed framework categorizes normal and abnormal behavior based on network data, and XGBoost optimizes trees through the use of effective split-finding algorithms to prevent overfitting and improve performance compared to conventional GB. The method also employs a simple baseline feature selection technique called Variance Threshold, which removes features with low variance.

Ullah et Mahmoud [11] introduced the IoTID20 dataset and applied various machine learning algorithms for classification and evaluation. The algorithms used included ensemble, Random Forest, Decision Tree, Gaussian NB, LDA, Logistic Regression, and Support-Vector Machines (SVM). To assess the performance of these models, the authors employed 3, 5, and 10-fold cross-validation measures. The results indicated that the Decision Tree, Random Forest, and Ensemble classifiers achieved the highest accuracy, while SVM and Logistic Regression had the lowest accuracy.

3. Methodology

This section outlines the details of the dataset, pre-processing methods, proposed machine learning models, and evaluation metrics.

To compare the performance of various machine learning algorithms for building an intrusion detection system (IDS), including decision tree (DT), random forest (RF), k-Nearest Neighbor (k-NN), Ada Boost, and support vector machine (SVM), using the IoTID20 dataset, we will follow the methodology outlined below:

- **Pre-processing:** Clean and prepare the dataset for use in machine learning algorithms. This may include handling missing values, performing feature selection or engineering, and scaling or normalizing the data as needed.
- **Feature selection:** is used to identify the most relevant variables for a predictive model
- **Splitting the data:** Divide the dataset into training and test sets, typically using a ratio of 80:20. The training set will be used to train the models, while the test set will be used to evaluate their performance.
- **Training and evaluating the algorithms:** For each of these algorithms (DT, RF, k-NN, Ada Boost, and SVM), we will train the model on the training data and then evaluate its performance on the test data. There are various metrics that we will use to evaluate the performance of the algorithms, such as accuracy, precision, recall, and F1-measure.
- **Comparing the results:** Once we have obtained the performance metrics for each of the algorithms, we will compare the results to determine which algorithm performs the best.

The overall framework of the system is shown in Fig. 1

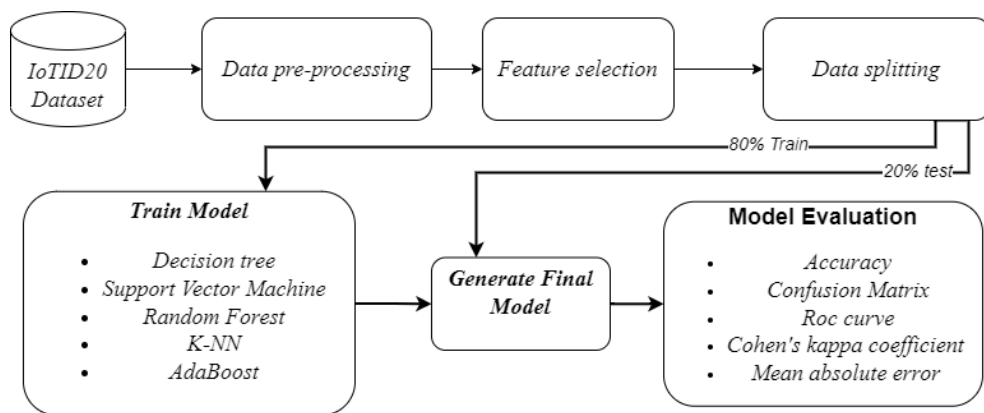


Fig. 1. The overall framework of the system

3.1. Dataset

Our experiment utilizes the IoTID20 dataset for network-based intrusion detection systems (IDS). This open-source dataset, which was collected from the IEEE Data Port and submitted by Huy Kang Kim [12], is labeled and includes a comprehensive network with various flow-based and general features specific to wireless environments. This dataset allows us to evaluate the ability to detect abnormal activity in IoT networks. It will serve as a benchmark for identifying anomalous activity across IoT networks and will be a valuable resource for the development of new IDS techniques for IoT environments. [11]. The IoTID20 dataset was generated using a home-connected smart device equipped with SKT NGU and EZVIZ Wi-Fi Smart camera (the IoT victim device), as well as other devices including laptops, smartphones, and tablets (IoT attacking devices). This dataset provides a realistic representation of IoT network activity, including both normal and abnormal behaviour [12].

This dataset contains 625,783 total instances with 83 IoT network features and three labels [10]: binary, category, and subcategory; details are given in Table 1. Fig 2 provides the IoTID20 attack taxonomy.

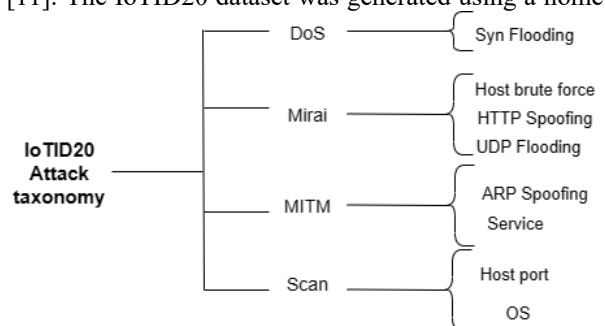


Fig. 2. IoTID20 attack taxonomy

Table 1. Distribution of attacks in IoTID20

Label	Count (Label)	Relative Frequency (Label)	Cat	Count (Cat)	Relative Frequency (Cat)	Sub_Cat	Count (Sub_Cat)	Relative Frequency (Sub_Cat)
Anomaly	585710	93,60%	Mirai	415677	66,40%	Mirai-UDP Flooding	183554	29,30%
Normal	40073	6,40%	Scan	75265	12%	Mirai-Hostbruteforceg	121181	1,94%
						DoS-Synflooding	59391	9,50%
						Mirai-HTTP Flooding	55818	8,90%
						Mirai-Ackflooding	55124	8,80%
						Scan Port OS	53073	8,50%
						Normal	40073	6,40%
						MITM ARP Spoofing	35377	5,70%
						Scan Hostport	22192	3,50%

3.2. Data pre-processing

Data pre-processing involves the preparation of raw data for a machine learning model. It is an essential step in the creation of a machine learning model, and it involves tasks such as data cleaning, label encoding, normalization, feature engineering, and data splitting. By performing these steps, we can ensure that the data is ready for use in a machine learning model.

3.2.1. Dataset Cleaning

Data cleaning involves identifying and correcting or removing incorrect, incomplete, and inaccurate data from datasets. It may also involve replacing missing values. In this experiment, we performed several data cleaning tasks, including removing duplicate rows, removing insignificant columns (such as 'Timestamp'), replacing infinite values with zero, and converting non-numerical features to numbers.

3.2.2. Label Encoding

Label encoding involves converting categorical labels into a numerical form that can be understood by machine learning algorithms. In this study, we used the label encoder approach to convert the categorical features 'Flow_Bytes/s', 'Flow_Pkts/s', 'Src_IP', and 'Dst_IP' into numerical values.

3.2.3. Feature selection

Feature selection is the process of identifying the most relevant input variables for a predictive model. By reducing the number of input variables, we can reduce the computational cost of modeling and potentially improve the model's performance. In this study, we used the low variance filter to eliminate features whose variance did not meet a threshold of 0.0 to 0.5. The original dataset had 83 features, and after applying the feature filtering approach, 62 features were selected. The choice of this particular threshold was motivated by the desire to strike a balance between retaining informative features and excluding those with limited discriminatory power.

3.2.4. Normalization

Normalization is the process of scaling the columns in a dataset to the same range while preserving the relative variations in the value ranges. In our case, the IoTID20 dataset has a variety of values, with some features having

negative values and others being in the thousands. To address this issue, we applied the min-max method to normalize the data between 0 and 1.

3.2.5. Data Splitting

Data splitting is the process of dividing a dataset into two or more subsets. One subset is usually used to train the model, while the other is used to evaluate the model's performance. In this study, we split the data into an 80% training set and a 20% testing set.

4. Experimental results

This section presents the results of the data analysis. In order to apply and evaluate our classifiers, we followed the pre-processing and preparation methods outlined in Section 3. We visually analyzed the results and used various evaluation criteria to compare the performance of the different classifiers.

All experiments on the Machine Learning algorithm described in this paper were conducted using KNIME analytics. KNIME Analytics Platform is a free, open-source software for data science. It allows you to uncover the insights hidden in your data, extract new knowledge, and make predictions about future trends. With its intuitive interface and powerful tools, KNIME makes it easy to explore and analyze your data [13]. KNIME provides a graphical interface for the entire development process, it contains various predefined nodes for machine learning tasks such as data pre-processing, classification, clustering and data visualizing.

4.1. Evaluation metrics

For evaluation, we consider the following metrics.

- Accuracy quantifies the percentage of correct predictions, given by:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

- Confusion Matrix tabulates correct and incorrect predictions made by a classification model.
- Precision calculates the ratio of correctly predicted positive outcomes to the total predicted positives:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- Recall computes the ratio of correctly predicted positive outcomes to all outcomes in a specific class:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- F1 Score is a combined metric of precision and recall, ranging from 0 to 1, defined as:

$$\text{F1 score} = \frac{2 \times (\text{recall} \times \text{precision})}{\text{recall} + \text{precision}}$$

- ROC is a graph showing a model's performance across various prediction thresholds, using true positive rate (TPR) and false positive rate (FPR):

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{FN}}$$

4.2. Performance of classifiers

We evaluate all classifiers in terms of accuracy, correctly classified, and incorrectly classified instances. The results are shown in Table 2.

Table 2. Performance of the classifiers

Evaluation criteria	Classifiers				
	DT	RF	K-NN	Ada Boost	SVM
Accuracy (%)	99.80	99.79	99.60	97.26	95.48
Correctly classified instances	55902	55897	55803	54482	53483
Incorrectly classified instances	113	118	212	1533	2532

4.3. Efficiency of classifiers

To check the efficiency of our classifiers, we compare the accuracy measures based on precision, recall, TP rate and FP rate values for DT, RF, Ada Boost, K-NN and SVM, as shown in Table 3.

Table 3. Comparison of accuracy measures for DT, RF, k-NN, Ada Boost, and SVM

	TP	FP	Precision	Recall	F-measure	class
DT	50478	65	0.999	0.999	0.999	Anomaly
	5424	48	0.991	0.988	0.990	Normal
RF	50517	109	0.998	0.999	0.999	Anomaly
	5380	9	0.998	0.980	0.989	Normal
K-NN	50454	140	0.997	0.999	0.998	Anomaly
	5349	72	0.987	0.974	0.981	Normal
Ada Boost	50401	1408	0.973	0.998	0.985	Anomaly
	4081	125	0.97	0.743	0.842	Normal
SVM	49355	1361	0.973	0.977	0.975	Anomaly
	4128	1171	0.779	0.752	0.765	Normal

Table 4. Confusion matrix

	Anomaly	Normal	class
DT	50478	48	Anomaly
	65	5424	Normal
RF	50517	9	Anomaly
	109	5380	Normal
K-NN	50454	72	Anomaly
	140	5349	Normal
Ada	50401	125	Anomaly
Boost	1408	4081	Normal
SVM	49355	1171	Anomaly
	1361	4128	Normal

To better understand efficiency, confusion matrix is used to evaluate our ML models. In Table 4, each row corresponds to actual class rates, while each column displays the predictions. In addition to the confusion matrix, we represent the ROC curve to illustrate the precision of each classifier. Fig. 3, Fig. 4, Fig. 5, Fig. 6, and Fig. 7 presents the ROC curve of our classifiers.

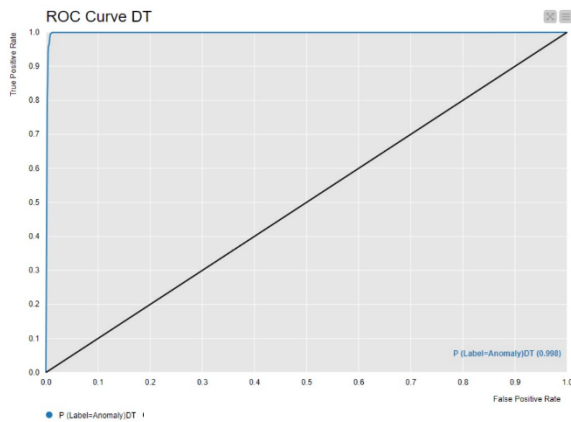


Fig. 3. ROC curve Decision tree

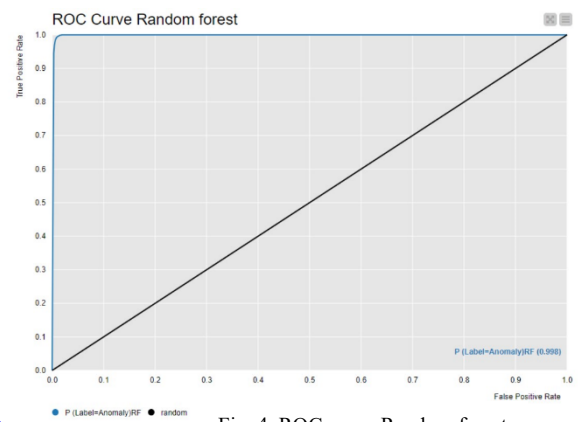


Fig. 4. ROC curve Random forest

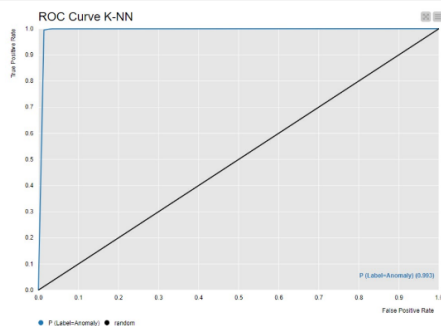


Fig. 5. ROC curve K-NN

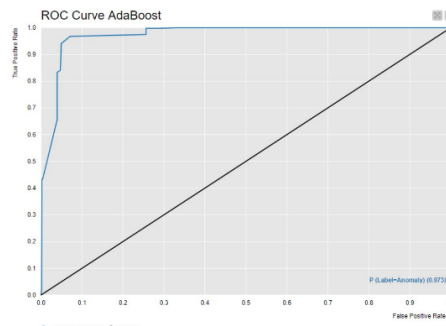


Fig. 6. ROC curve Ada Boost

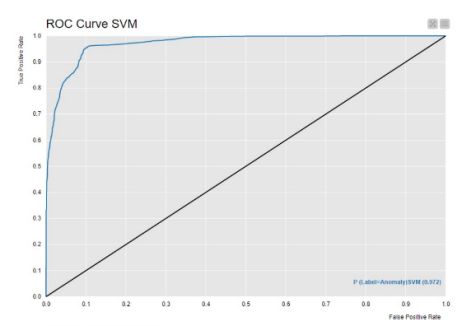


Fig. 7. ROC curve SVM

4.4. Comparison with related research

Table 5 summarize the result of our work in comparison with existing related research in term of accuracy.

Table 5. Description of related IoT attack detection.

Author	Dataset	Classification Type	Algorithm	Evaluation Metric
Hasan et al [5]	DS20S traffic traces	Multiclass	Random Forest ANN	ACC = 99.4 ACC = 99.4
Jingyi Su et al. [6]	IoTID20	Binary	Decision tree GBM Random Forest	ACC = 97.83 ACC = 97.84 ACC = 96.36
Reddy et al [7]	IoTID20	Binary	Decision tree Random forest k-NN GB XGBoost	ACC=99.91 ACC=99.92 ACC=99.3 ACC=99.89 ACC=99.96
Ullah et mahmoud [8]	IoTID20	Multiclass	Decision tree Random Forest Ensemble	ACC = 88 ACC = 84 ACC = 87
Our work	IoTID20	Binary	Decision tree Random forest k-NN AdaBoost SVM	ACC=99.80 ACC=99.79 ACC=99.60 ACC=97.26 ACC=95.48

5. Discussion and comparison

After building the predictive model, we can assess the performance of our algorithms by analyzing the results. As shown in Table 2, the decision tree (DT) and random forest (RF) algorithms had the highest accuracy (99.80% and 99.79% respectively) on the IoTID 20 dataset, with the DT algorithm performing slightly better than the RF algorithm. The k-nearest neighbors (K-NN) algorithm also had a high accuracy, but it was slightly lower than the DT and RF algorithms. The Ada Boost algorithm had a lower accuracy than the other three algorithms, but it was still relatively high. The support vector machine (SVM) algorithm had the lowest accuracy of the five algorithms. Additionally, Figure 2 illustrates that the decision tree classifier performs better than the other classifiers, with the highest number of correctly classified instances and the lowest number of incorrectly classified instances.

Table 4 is about the confusion matrix of the classifier, revealing that both the random forest and decision tree classifiers have the highest true positive (TP) values for the anomaly class, with 50517 and 50478 instances respectively. However, the SVM algorithm correctly identifies 49355 instances belonging to the anomaly class. The false positive (FP) rate is lower for the decision tree and random forest classifiers (65 and 109 instances, respectively, for the anomaly class, and 48 and 9 instances, respectively, for the normal class), followed by the k-NN Ada Boost, and SVM algorithms. These results explain why the decision tree and random forest classifiers outperform the other classifiers.

Now, let's compare actual class and predicted results obtained using the confusion matrix depicted in Table 4. Decision tree correctly predicts 55902 instances out of 56015 instances and makes 113 incorrect predictions. That is why the accuracy of the Decision tree is better than other classification techniques used with lower error rate value.

The ROC curve graph in Fig. 3, Fig. 4, Fig. 5, Fig.6, and Fig.7, illustrates the performance of different machine learning algorithms. It is clear that the Decision tree and Random forest algorithms are excellent classifiers, as they

start from the left corner and reach the upper left and upper right corners with a sensitivity of 99% and a specificity of 98%. The K-NN and SVM algorithms also demonstrate good performance, but are not as effective as Decision tree and Random forest.

Table 5 summarizes the comparison results of our work with existing related research in terms of accuracy. Our experimental results reveal that the Decision tree algorithm achieved the highest accuracy at 99.80%. In contrast, the SVM algorithm had the lowest accuracy at 95.48%. In comparison to other research on intrusion detection using IoT datasets that have evaluated the classification accuracies of different data mining algorithms, our results show that the Decision tree approach consistently produced optimal results when compared through accuracy, sensitivity, specificity, and precision metrics.

6. Conclusion

In this study, we evaluated the effectiveness of five machine learning algorithms - Decision tree, Random Forest, k-NN, Ada Boost, and SVM - for detecting intrusions on IoTID20 dataset. We compared the algorithms based on accuracy, sensitivity, precision, and specificity in order to determine the best classification accuracy. The Decision tree and Random Forest algorithms achieved the highest accuracy of 99.80% and 99.79%, respectively, outperforming the other algorithms. Our results demonstrate that the Decision tree and the Random Forest are effective at predicting IoT intrusions and have the best performance in terms of accuracy and precision.

References

- [1] M. Abualkibash, "Machine Learning in Network Security Using KNIME Analytics," *Int. J. Netw. Secur. Its Appl.*, vol. 11, no. 5, pp. 1–14, Sep. 2019, doi: 10.5121/ijnsa.2019.11501.
- [2] M. Arafat, A. Jain, and Y. Wu, "Analysis of intrusion detection dataset NSL-KDD using KNIME analytics," *Proc. 13th Int. Conf. Cyber Warf. Secur. ICCWS 2018*, vol. 2018-March, pp. 573–583, 2018.
- [3] S. B. Kotsiantis, "Decision trees: a recent overview," *Artif. Intell. Rev.*, vol. 39, no. 4, pp. 261–283, Apr. 2013, doi: 10.1007/s10462-011-9272-4.
- [4] W. S. Noble, "What is a support vector machine?," *Nat. Biotechnol.*, vol. 24, no. 12, pp. 1565–1567, Dec. 2006, doi: 10.1038/nbt1206-1565.
- [5] G. Biau and E. Scornet, "A random forest guided tour," *TEST*, vol. 25, no. 2, pp. 197–227, Jun. 2016, doi: 10.1007/s11749-016-0481-7.
- [6] Y. Freund and R. E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997, doi: 10.1006/jcss.1997.1504.
- [7] O. Kramer, "K-Nearest Neighbors," 2013, pp. 13–23. doi: 10.1007/978-3-642-38652-7_2.
- [8] M. Hasan, M. M. Islam, M. I. I. Zarif, and M. M. A. Hashem, "Attack and anomaly detection in IoT sites using machine learning approaches," *Internet of Things*, vol. 7, p. 100059, Sep. 2019, doi: 10.1016/j.iot.2019.100059.
- [9] J. Su, S. He, and Y. Wu, "Features selection and prediction for IoT attacks," *High-Confidence Comput.*, vol. 2, no. 2, p. 100047, Jun. 2022, doi: 10.1016/j.hcc.2021.100047.
- [10] D. K. K. Reddy, H. S. Behera, J. Nayak, B. Naik, U. Ghosh, and P. K. Sharma, "Exact greedy algorithm based split finding approach for intrusion detection in fog-enabled IoT environment," *J. Inf. Secur. Appl.*, vol. 60, no. June, p. 102866, 2021, doi: 10.1016/j.jisa.2021.102866.
- [11] I. Ullah and Q. H. Mahmoud, "A Technique for Generating a Botnet Dataset for Anomalous Activity Detection in IoT Networks," vol. 2020-Octob, no. May. Springer International Publishing, 2020. doi: 10.1109/SMC42975.2020.9283220.
- [12] H. Kang, D. H. Ahn, G. M. Lee, J. Do Yoo, K. H. Park, and H. K. Kim, "IoT network intrusion dataset," *IEEE Dataport*, 2019, [Online]. Available: <https://iee-dataport.org/open-access/iot-network-intrusion-dataset>
- [13] "KNIME Getting Started Guide." <https://www.knime.com/getting-started-guide>
- [14] Christin Alex, Giselle Creado, Wesam Almobaideen, Orieb Abu Alghanam, Maha Saadeh, "A Comprehensive Survey for IoT Security Datasets Taxonomy, Classification and Machine Learning Mechanisms," *Computers & Security*, Volume 132, 2023, 103283, ISSN 0167-4048, doi: 10.1016/j.cose.2023.103283
- [15] Rasheed Ahmad, Izzat Alsmadi, Wasim Alhamdani, Lo'ai Tawalbeh, "A comprehensive deep learning benchmark for IoT IDS," *Computers & Security*, Volume 114, 2022, 102588, ISSN 0167-4048, doi: 10.1016/j.cose.2021.102588.
- [16] Nuno Prazeres, Rogério Luís de C. Costa, Leonel Santos, Carlos Rabadão, "Engineering the application of machine learning in an IDS based on IoT traffic flow," *Intelligent Systems with Applications*, Volume 17, 2023, 200189, ISSN 2667-3053,